

Projeto 1: Algoritmos para Grafos

Minimização do Flowtime no Problema Open Shop

Gabriel Davi; Giordani Andre

Professor: Tadeu Zubaran

GitHub Link: <https://github.com/GabrielDavi7/OpenShop-Flowtime>

6 de maio de 2026

1 Introdução e Definição do Problema

Este relatório descreve a implementação de uma solução baseada em grafos para o problema de agendamento *Open Shop* (OSSP), focando na minimização do *Flowtime* ($\sum C_j$) [1].

O problema OSSP é um dos desafios mais rigorosos da Pesquisa Operacional, classificado como NP-Completo por Gonzalez e Sahni (1976) [2]. A sua complexidade reside na liberdade de ordem das máquinas para cada tarefa, o que expande o espaço de busca de forma exponencial. A minimização do *Flowtime* é essencial para reduzir o tempo médio de permanência dos trabalhos no sistema.

2 Fundamentação Teórica e Estado da Arte

A construção da solução inicial (Baseline) seguiu os critérios de Bräsel et al. (2008) [3], utilizando a heurística SPT (*Shortest Processing Time*). Bräsel demonstra que, embora eficaz, esta abordagem estagna em matrizes "quadradas" ($n = m$), onde os conflitos entre recursos são máximos.

Para superar esta barreira, a literatura, como o trabalho de Andresen et al. (2008) [4], propõe meta-heurísticas iterativas. No entanto, Andresen relata que algoritmos sequenciais enfrentam dificuldades de estagnação em instâncias densas de Taillard. O diferencial deste projeto é a utilização de uma **Busca Tabu Paralela com VNS**, permitindo que o sistema explore o grafo através de 12 *threads* simultâneas, quebrando a barreira da estagnação reportada na literatura clássica.

3 Desenvolvimento e Ferramentas de Otimização

O motor de busca foi refinado de forma incremental através da integração de diversas técnicas:

- **Busca Local e Vizinhanças:** Operadores de *Swap* e *Insertion* aplicados sobre as arestas disjuntivas do DAG para ajustes finos no caminho crítico.
- **VNS com Estratégia de Paciência:** Entre o desenvolvimento da busca local e a implementação da Busca Tabu, introduzimos o framework *Variable Neighborhood Search*. Um refinamento crucial foi o parâmetro de "**Paciência 2000**": caso o algoritmo realize 2000 iterações consecutivas sem qualquer melhoria no *Flowtime*, é disparado um movimento de *Shaking* agressivo para deslocar a busca para uma nova região do espaço de soluções.

- **Busca Tabu:** Implementámos uma memória de curto prazo (Lista Tabu) para proibir movimentos redundantes, garantindo que o algoritmo não ficasse preso em ciclos viciosos.
- **Paralelismo Massivo (12 Threads):** Disparo de 12 buscas independentes em paralelo via *std::thread*, maximizando o uso do hardware moderno (i5-11400H).

4 Análise de Resultados Computacionais

4.1 Convergência e Teto de Melhoria

Um dado fundamental observado foi o comportamento do algoritmo em relação ao tempo. Para as instâncias de menor dimensão (4×4 e 5×5), realizámos testes de estresse variando o tempo limite entre **20, 40 e 60 segundos**, estendendo-se até **6 minutos**.

Concluimos que, para estas instâncias, **não houve qualquer diferença nos resultados**, independentemente do tempo adicional investido. Isto demonstra que o algoritmo atinge o seu **"teto de melhoria"** (provável ótimo global) quase instantaneamente nessas dimensões. O tempo de 6 minutos foi, portanto, mantido para garantir que apenas as instâncias massivas (20×20) pudessem explorar vizinhanças mais profundas onde a complexidade é maior.

4.2 Otimização do Flowtime e Resultados Completos

A Tabela 1 apresenta a comparação entre a Baseline e o resultado após a otimização paralela exaustiva em instâncias selecionadas de alto impacto.

Instância	Tamanho	Baseline (Flow)	VNS Paralelo (Flow)	Melhoria (%)
ta010sp	4×4	1043	958	-8,15%
ta100sp	4×4	1242	982	-20,93%
ta160sp	5×5	2482	1924	-22,48%
ta220sp	7×7	5021	3704	-26,23%
ta310sp	10×10	9254	8915	-3,66%
ta510sp	20×20	39392	33272	-15,54%

Tabela 1: Impacto da Busca Tabu Paralela na redução do Flowtime.

Resultados Completos: Os dados de todas as 60 instâncias processadas podem ser consultados no link:

<https://docs.google.com/spreadsheets/d/10CVjgphW9PZcB0Gh2DISvuyNJWrwXtZXmzWTLuSP9ko/edit?usp=sharing>

4.3 Análise de Consistência por Escala

Para validar a robustez da arquitetura além de instâncias isoladas, os resultados foram consolidados em médias por agrupamento de tamanho (Tabela 2). Observa-se que o algoritmo manteve consistência em todos os cenários, atingindo o pico de maturação de otimização operacional nas matrizes 7×7 (-17,00%). Mesmo nas instâncias massivas de alta densidade de conflito (20×20), obteve-se uma redução sólida aproximada de 7,5%.

Tamanho da Instância	Melhoria Média no Flowtime (%)
4×4	-8,73%
5×5	-12,91%
7×7	-17,00%
10×10	-10,55%
15×15	-7,79%
20×20	-7,49%

Tabela 2: Média global de redução do Flowtime agrupada por escala de matriz.

4.4 Sinergia e Trade-off com o Makespan

A minimização do *Flowtime* gerou um efeito de otimização colateral no *Makespan* em instâncias de menor complexidade. A Tabela 3 apresenta o impacto médio no *Makespan* agrupado por escala, destacando reduções significativas em matrizes de até 10×10 .

No entanto, o verdadeiro *trade-off* surge nas matrizes de maior porte (15×15 e 20×20), onde o algoritmo realiza uma piora controlada no *Makespan* (+0,07% e +0,70%, respectivamente) para garantir o benefício global no fluxo. A Tabela 4 ilustra instâncias individuais, com destaque para a **ta600sp**, onde o *Makespan* subiu +9,17% refletindo o sacrifício estratégico do gargalo.

Tamanho da Instância	Melhoria/Piora Média no Makespan (%)
4×4	-10,70%
5×5	-16,50%
7×7	-15,36%
10×10	-7,16%
15×15	+0,07%
20×20	+0,70%

Tabela 3: Média global de impacto no Makespan agrupada por escala de matriz (valores positivos indicam piora).

Instância	Baseline (Makespan)	Otimizado (Makespan)	Impacto (%)
ta010sp	352	302	-14,20%
ta100sp	412	322	-21,84%
ta160sp	721	506	-29,82%
ta220sp	953	782	-17,94%
ta510sp	2634	2473	-6,11%
ta600sp	2649	2892	+9,17%

Tabela 4: Impacto secundário no Makespan em instâncias específicas.

5 Conclusão

O projeto demonstrou que a modelagem por grafos aliada ao processamento paralelo e estratégias de "paciência" no VNS permite quebrar as barreiras de estagnação da literatura clássica. Os testes de tempo provaram que o algoritmo atinge o seu teto de eficiência rapidamente em

instâncias pequenas, enquanto as reduções médias consistentes em todas as escalas e os picos de até **26,23%** em instâncias individuais validam a extrema eficácia da abordagem.

Referências

- [1] ZUBARAN, Tadeu. *Especificação do Projeto 1: Algoritmos para Grafos*. 2026.
- [2] GONZALEZ, T.; SAHNI, S. *Open Shop Scheduling to Minimize Finish Time*. JACM, 1976.
- [3] BRÄSEL, H. et al. *Heuristic constructive algorithms for open shop scheduling....* EJOR, 2008.
- [4] ANDRESEN, M. et al. *Simulated annealing and genetic algorithms for minimizing mean flow time....* MCM, 2008.